

EstetoOne API Documentation

HEN-25 — OpenAPI / Swagger deliverable

Generated from estetoone-backend OpenAPI 3.x specification

1. Environment setup

Backend (estetoone-backend):

- `cp .env.example .env`
- `npm install && npx prisma migrate deploy && npm run prisma:seed`
- `npm run start:dev` → `http://localhost:3000`

Frontend (stetho-ai-connect-25039, optional):

- `npm install && npm run dev` → `http://localhost:8080`
- Dev hub: `http://localhost:8080/dev/api-testing`

2. Swagger UI access

Interactive docs with Try-it-out:

URL: `http://localhost:3000/api/docs`

OpenAPI JSON: `http://localhost:3000/api/docs-json`

Workflow:

1. Open Swagger UI in browser
2. POST `/v1/auth/login` with seed credentials (see section 5)
3. Copy `accessToken` → Authorize → Bearer token
4. Use Try-it-out on any endpoint

3. Endpoint inventory (51 operations)

Domain	Method	Path	Auth	Summary
auth	GET	<code>/v1/auth/check-email</code>	Public	Check if email is available for registration
auth	POST	<code>/v1/auth/login</code>	Public	Login with email and password
auth	POST	<code>/v1/auth/logout</code>	Cookie (refresh_token)	Logout and clear refresh

			n)	cookie
auth	DELETE	/v1/auth/me	Bearer JWT	Delete current user account
auth	GET	/v1/auth/me	Bearer JWT	Get current authenticated user
auth	POST	/v1/auth/refresh	Cookie (refresh_token)	Refresh access token using refresh_token cookie
auth	POST	/v1/auth/register	Public	Register a new user (sets refresh_token cookie)
auth	POST	/v1/auth/validate-registration	Public	Validate registration payload before signup
consultations	POST	/v1/consultations/instant/match	Bearer JWT	Doctor accepts next queued instant consultation
consultations	POST	/v1/consultations/instant/request	Bearer JWT	Request instant consultation
consultations	GET	/v1/consultations/me	Bearer JWT	List consultations for current role
consultations	GET	/v1/consultations/me/history	Bearer JWT	Get consultation history
consultations	POST	/v1/consultations/schedule	Bearer JWT	Schedule a consultation
consultations	GET	/v1/consultations/{id}/session	Bearer JWT	Get video session room URL
consultations	POST	/v1/consultations/{id}/session	Bearer JWT	Create/start video session
consultations	PATCH	/v1/consultations/{id}/status	Bearer JWT	Update consultation status

doctor-dashboard	GET	/v1/doctor-dashboard/appointments/today	Bearer JWT	Get today's appointments
doctor-dashboard	PATCH	/v1/doctor-dashboard/online-status	Bearer JWT	Toggle doctor online status
doctor-dashboard	GET	/v1/doctor-dashboard/patients	Bearer JWT	List linked patients
doctor-dashboard	POST	/v1/doctor-dashboard/patients	Bearer JWT	Link patient to doctor
doctors	GET	/v1/doctors	Public	Search doctors (paginated)
doctors	GET	/v1/doctors/me/availability	Bearer JWT	Get own availability schedule
doctors	PATCH	/v1/doctors/me/availability	Bearer JWT	Merge availability schedule
doctors	PUT	/v1/doctors/me/availability	Bearer JWT	Replace availability schedule
doctors	GET	/v1/doctors/online	Public	List online doctors (paginated)
doctors	GET	/v1/doctors/{id}	Public	Get doctor detail by profile id
health	GET	/v1/health	Public	Aggregate health status (always 200)
health	GET	/v1/health/live	Public	Liveness probe
health	GET	/v1/health/ready	Public	Readiness probe (DB + Redis)
health-wallet	GET	/v1/health-wallet/me	Bearer JWT	Get aggregated health wallet
health-wallet	GET	/v1/health-wallet/me/consultations	Bearer JWT	List consultation records for health wallet
health-wallet	GET	/v1/health-wallet/me/vitals	Bearer JWT	List patient vitals

health-wallet	POST	/v1/health-wallet/me/vitals	Bearer JWT	Record a vital sign
medical-records	GET	/v1/medical-records/me	Bearer JWT	List patient medical records
medical-records	POST	/v1/medical-records/me	Bearer JWT	Create medical record
medical-records	DELETE	/v1/medical-records/me/{id}	Bearer JWT	Delete medical record
medical-records	GET	/v1/medical-records/me/{id}	Bearer JWT	Get medical record detail
payments	GET	/v1/payments/plans	Public	List payment plans
payments	POST	/v1/payments/setup-intent	Bearer JWT	Create Stripe setup intent (mocked without STRIPE_SECRET_KEY)
payments	GET	/v1/payments/subscription	Bearer JWT	Get current user subscription
payments	POST	/v1/payments/webhook	Stripe signature	Stripe webhook handler (raw body + signature)
plans	GET	/v1/plans	Public	List available subscription plans
plans	GET	/v1/plans/{planId}/validate	Public	Validate plan availability by id
profiles	GET	/v1/profiles/doctor/me	Bearer JWT	Get doctor profile
profiles	PATCH	/v1/profiles/doctor/me	Bearer JWT	Update doctor profile
profiles	DELETE	/v1/profiles/doctor/me/photo	Bearer JWT	Delete doctor profile photo
profiles	PUT	/v1/profiles/doctor/me/photo	Bearer JWT	Upload doctor profile photo

profiles	GET	/v1/profiles/patient/me	Bearer JWT	Get patient profile
profiles	PATCH	/v1/profiles/patient/me	Bearer JWT	Update patient profile
profiles	DELETE	/v1/profiles/patient/me/photo	Bearer JWT	Delete patient profile photo
profiles	PUT	/v1/profiles/patient/me/photo	Bearer JWT	Upload patient profile photo

4. Examples by domain

auth

Endpoint: POST /v1/auth/login

Request: {"email":"maria.silva@example.com","password":"password123"}

Response: {"accessToken":"...","user":{"id":"...","email":"...","role":"patient"}}

health

Endpoint: GET /v1/health/live

Request: (no body)

Response: {"status":"ok","timestamp":"..."}

plans

Endpoint: GET /v1/plans

Request: (no body)

Response: [{"id":"basic","name":"Basic",...}]

consultations

Endpoint: POST /v1/consultations/schedule

Request: {"doctorId":"<uuid>","scheduledAt":"2026-12-01T10:00:00.000Z"}

Response: {"id":"...","status":"waiting","type":"SCHEDULED",...}

payments

Endpoint: POST /v1/payments/setup-intent

Request: (Bearer required, no body)

Response: {"clientSecret":"seti_mock_...","setupIntentId":"seti_mock_..."}

5. Seed test credentials

- Patient: maria.silva@example.com / password123
- Doctor: dr.carlos@example.com / password123

6. Mocked integrations & known gaps

- Stripe: MockStripeClient when STRIPE_SECRET_KEY is empty
- Daily.co video: returns https://mock.daily.co/room-`{consultationId}`
- S3 documents: stored inline in database (s3Key column)
- CORS: not enabled in backend — use Swagger on :3000 or configure proxy
- Regenerate spec: npm run openapi:export in estetoone-backend